

# Python编程

## 开启你的代码魔法之旅

✨ 一行代码，玩转世界！

主讲老师：黄礼泊

助教：张蔚

黄礼泊

博士，助理研究员  
中国科学院计算技术研究所

### 科学研究成果

聚焦于机器学习与神经科学交叉领域，在相关领域的国际顶级期刊、会议（包括Nature子刊、IEEE Transactions系列会刊，人工智能国际顶级会议ICML、LCLR、NeurIPS等）发表论文30余篇，授权、受理专利10余项。

### 社会任职与项目

国际IEEE学会、中国计算机学会、人工智能学会、指挥与控制学会专委会委员/会员；承担多项国家级项目。

### 教育学习背景

2019/10-2020/10：Brunel University of London，  
EEE，联合培养博士

2018/9-2021/7：广东工业大学，信息与通信工程，  
工学博士

2016/9-2018/7：广东工业大学，计算机科学与技术，  
工学硕士

2012/9-2016/7：江西师范大学，数学，理学学士

# 你好，未来的魔法师！

欢迎来到代码的世界！在这里，Python 将成为你的魔法杖



## 精准的内容推送

小视频如何找到你？算法在背后默默工作



## 酷炫的游戏引擎

角色奔跑、战斗与特效，由代码逻辑驱动

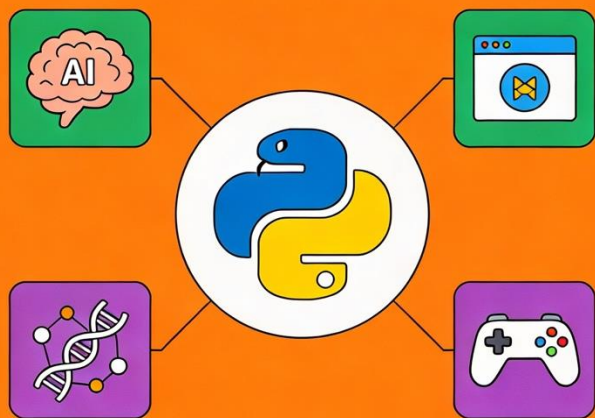


## 太空数据处理

NASA如何分析火星探测器传回的信息？

答案揭晓：驱动这一切的魔法核心就是——Python！

# Python: 不止是一条蛇



Python: 编程界的万能瑞士军刀

它非常**简单、优雅**，而且功能超级强大！从AI机器人到太空探索，无所不能。



## AI与大数据

开发聊天机器人与自动驾驶核心算法



## 网站与APP

B站、知乎等大厂后台的首选语言



## 科学与航天

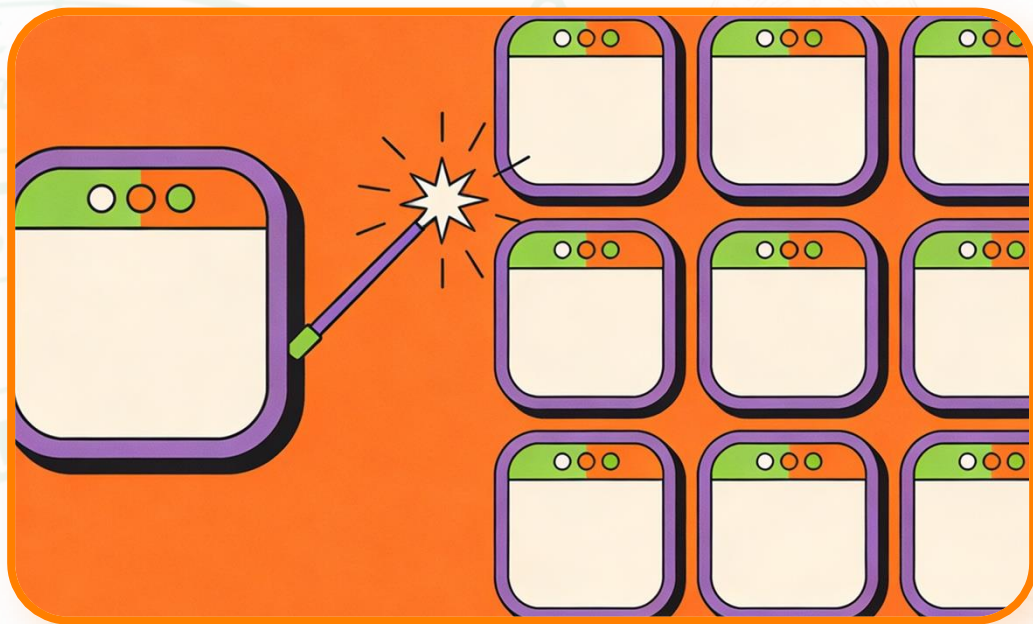
NASA用它分析太空数据与气候模拟



## 游戏开发

《我的世界》模组开发的得力助手

# 见证奇迹的时刻！一行代码的魔力



💡 魔法原理：用简单指令触发批量复制

编程很难？写很多行代码才有效？No！在Python的世界里，有时候一行代码就能创造奇迹，就像施展魔法一样简单！



```
print("Hello World!" * 10)
```

🌟 效果展示：将短语瞬间复制10次并打印！这就是Python简洁而强大的魅力，让计算机成为你的魔法助手。

# 你的魔法工具箱准备好了吗？



✨ 从左至右：PyCharm / VS Code / 在线编辑器



## PyCharm：屠龙宝刀

专业级IDE，功能强大，适合进阶开发。



## VS Code：瑞士军刀

轻量级编辑器，颜值高，插件丰富灵活。



## 在线编辑器：便捷之选

无需安装，打开网页即用，适合新手入门。



现在行动：请检查电脑，确认IDE已安装并能正常打开！

## 01 课程基础



# 基础：魔法盒与传声筒



### 魔法盒 (变量)

存储信息的容器，像百宝箱一样，让程序随时记住关键数据。



### 传声筒 (输入输出)

与程序对话的桥梁，你向它下达指令，它为你展示运行结果。

🌟 掌握这两个基础道具，开启你的编程魔法之旅！ 🌟

# 变量(Variables): 给数据一个家



🌟 魔法收纳盒：存放数据的容器

## 你的专属魔法收纳盒

在编程世界里，信息（如名字、年龄）需要临时存放。Python中的“变量”就像一个贴了标签的收纳盒，随时取用。

```
name="哈利波特"  
hp=100
```

💡 **注意：**等号“=”不是数学的“等于”，而是“赋值”——把右边的东西放进左边的盒子里。

核心要点：变量 = 贴标签的盒子 + 里面的数据



# 如何给你的魔法盒起名字？——变量的命名规则

“给变量起名字可不是随便乱起的，要遵守一些规则哦！”



## 字符规范

只允许包含：字母、数字和下划线（\_）



## 首字符限制

不能以数字开头！如  
123name不行，但name123  
可以。



## 区分大小写

Python区分大小写，Name  
和name是两个完全不同的  
盒子！



## 避开关键字

严禁使用if, else, while等  
Python保留字，它们有特殊含义。

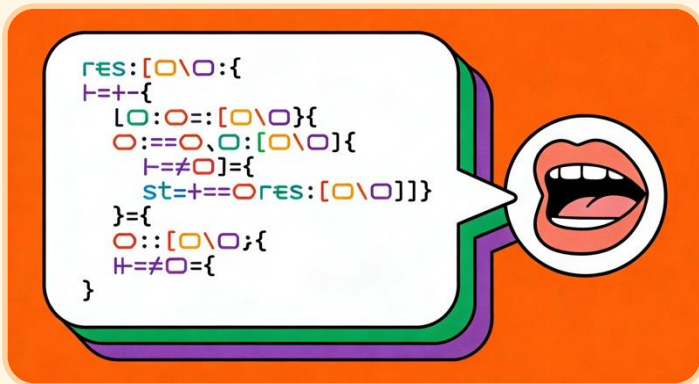


## 见名知意

尽量使用有意义的名字，  
如用age代表年龄，代码更  
易懂。

小提示：规范的命名是编写高质量代码的第一步！

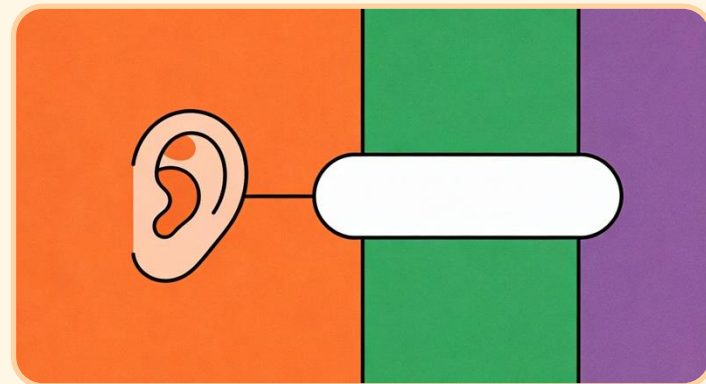
# Input/Output: 程序的嘴巴和耳朵



## ✓ print() - 程序的嘴巴（传声筒）

像传声筒一样，将程序内部计算好的信息“大声”地输出展示给我们。

```
print("我是一个会说话的程序！")
```



## 📢 input() - 程序的耳朵（接收器）

像接收器一样，程序会暂停运行，等待用户输入指令或数据后继续。

```
user_input = input("请告诉我你的名字：")
```

# 动手实践：身份登记表

## 小练习



### 任务目标：收集信息

1. 询问用户的姓名 (使用 input)
2. 询问用户的年龄 (使用 input)
3. 打印个性化欢迎语 (使用 print & f-string)

💡 提示：记得给变量起有意义的名字哦！

## Python 终端

```
# 获取用户输入
name= input("嗨！请问你叫什么名字？")
age= input("你今年几岁了？")
# 打印欢迎信息
print(f'欢迎来到 2026 年，{name}！你今年 {age} 岁了。')
```

## 02 课程进阶

# 函数：编程的“魔法工厂”

从重复的代码中解放出来，学习如何创建可复用的“魔法工厂”，让你的代码更简洁、更强大。



### 封装复用

将常用功能打包成独立模块，避免重复造轮子，提升效率。



### 参数传递

灵活接收外部数据作为“原材料”，实现动态计算与逻辑控制。



### 返回结果

处理完成后输出“产品”，将计算结果反馈给调用者，形成闭环。

# 什么是函数？为什么需要函数？



## 核心思想：自动售卖机

**定义：**接收输入(参数)，执行操作，返回输出(结果)的可复用代码块。

 就像自动售卖机：投入硬币(参数)，机器内部操作，吐出饮料(返回值)。



## 核心价值：效率与管理

**优势：**避免重复造轮子，将复杂系统拆解为独立模块，降低复杂度。

- 代码复用：一次编写，多处调用
- 抽象化：隐藏细节，专注逻辑

# 函数的结构：如何定义一个“魔法工厂”

## 语法结构：

```
def 函数名(参数1, 参数2...):  
    # 函数体代码  
    return 返回值
```

## 示例：

```
def greet(name):  
    msg = f"你好, {name}!"  
    return msg
```

定义语法 · 逻辑封装 · 结果返回



## def 关键字 & 函数名

def 声明定义，函数名是工厂的招牌，方便后续调用执行。



## 参数 (Parameters)

工厂的“原材料”，函数的输入数据。  
可以有多个参数，也可以没有参数。



## 函数体 & return 返回值

函数体是具体的生产流程代码。  
return 用于将生产好的“产品”交付出来。



一句话总结：函数是封装逻辑的工厂，参数是原料，返回值是产品

# 调用函数：启动你的“魔法工厂”



## 函数调用语法

变量 = 函数名(参数1, 参数2, ...)



## 代码示例

```
result = greet("小明") # 调用函数并传参  
print(result) # 输出：你好，小明！欢迎来到代码魔法学院。
```



## 调用执行过程

1. 跳转定义 → 2. 参数赋值 → 3. 执行代码 → 4. return返回 → 5. 结果赋值给变量

# 动手实践：创建一个加法函数

## 任务目标

定义一个名为**add**的函数，它接收两个数字作为参数，返回它们的和。

## 操作提示

- 使用**def**关键字定义函数。
- 参数设置为**a**和**b**。
- 在函数体中计算**a + b**。
- 使用**return**返回计算结果。

## 代码示例

参考以下代码，定义函数并进行调用，观察控制台输出结果。

```
# 定义加法函数
def add(a, b):
    sum_result = a + b
    return sum_result

result = add(3, 5)
print(f"3 + 5 = {result}")
```

### 03 核心进阶



## 决策：分岔路口的逻辑

从简单的顺序执行，进阶到赋予程序“思考”的能力  
探索如何在编程中处理不同情况，做出正确选择

# if/else: 程序的大脑 —— 条件判断逻辑



💡 生活中的岔路口 = 程序的判断逻辑

生活中我们总面临选择（如下雨带伞），程序也一样。它通过**条件判断**机制，根据不同情况做出不同反应。

```
score = 75
if score > 60:
    print("恭喜通过！")
else:
    print("遗憾，需重修")
```

程序会先检查**条件**（如分数>60）：

✅ 成立 → 执行 if 代码块 | ❌ 不成立 → 执行 else 代码块

# 比较运算符：天平的两端

做出判断的工具：像天平一样精准比较数值关系

▲ **> 大于**

左侧数值严格大于右侧

▮ **>= 大于等于**

左侧数值大于或等于右侧

⋮ **== 等于**

判断左右两边数值是否相等

▼ **< 小于**

左侧数值严格小于右侧

⋮ **<= 小于等于**

左侧数值小于或等于右侧

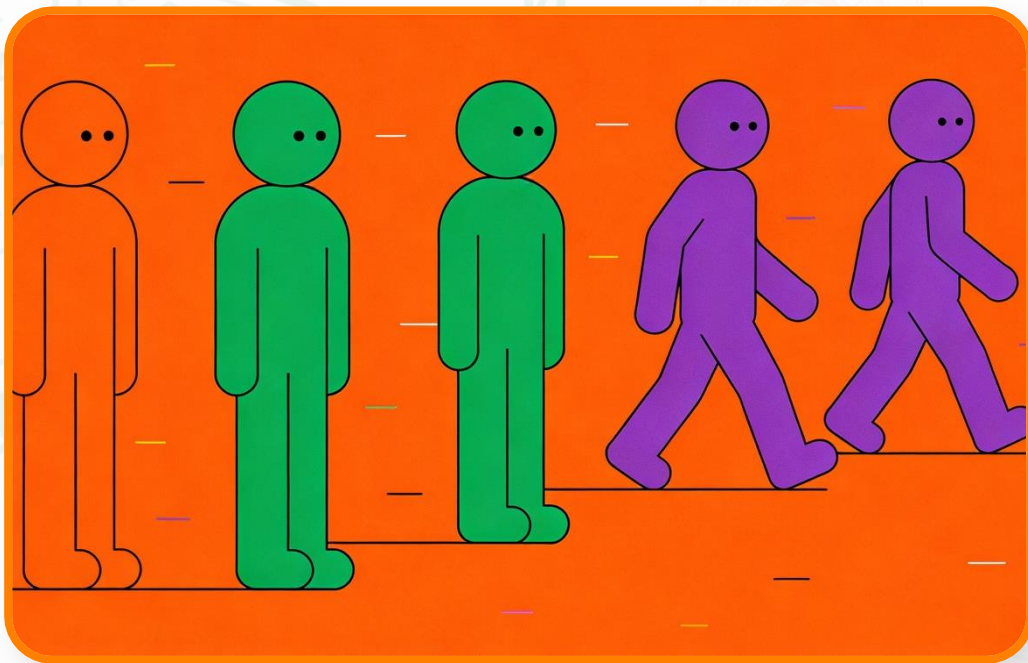
⊗ **!= 不等于**

判断左右两边数值不相等



**核心易错点：**单个等号“=”是赋值，两个等号“==”才是判断相等！

# 缩进：Python的灵魂！



缩进就像排队，对齐才整齐！

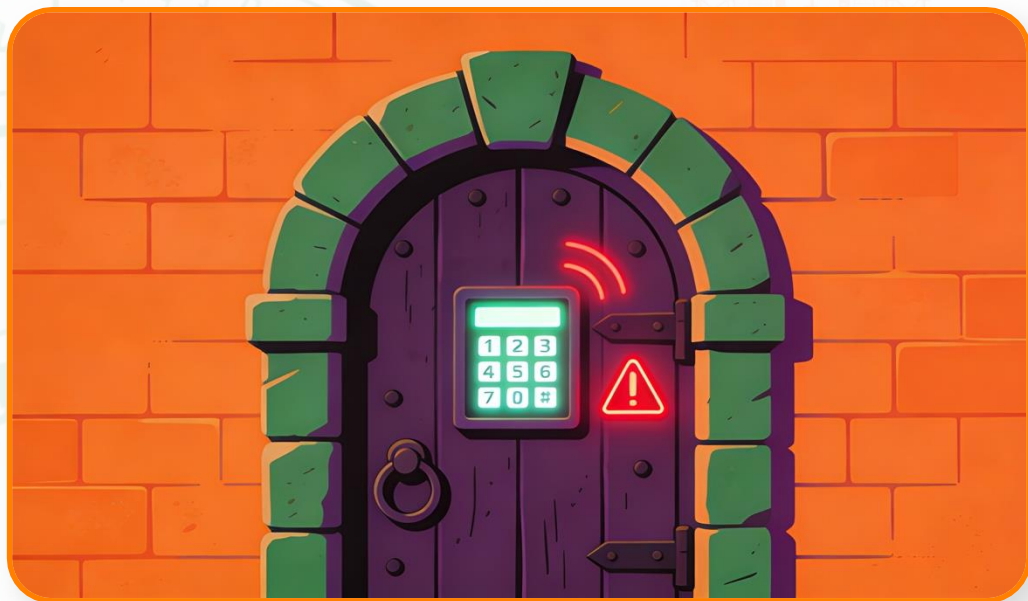
## Python的“颜值即正义”

Python用缩进来区分代码块（不像其他语言用{}）。可以把缩进想象成“家庭关系”：同一个缩进级别的代码，就是一家人。

```
if 条件成立:  
    print("我是孩子，必须缩进")  
print("我和if是平级的，不缩进")
```

⚠ 警告：不缩进就报错！

# 挑战：破解密室大门



▲ 密室大门实景：口令验证终端



## 任务：智能门禁验证

输入“芝麻开门”解锁成功，否则触发红色警报。用 `input` 获取输入，`if/else` 判断结果。



## Python 核心逻辑代码

```
password = input("请输入开门口令：")
if password == "芝麻开门":
    print("恭喜开门!")
else:
    print("警报！非法闯入！")
```

## 04 编程进阶



# 循环：永不疲惫的机器人

💡 核心概念：循环结构能让程序像机器人一样，高效自动地完成重复任务！

# while循环：不知疲倦的机器人



核心逻辑：条件判断 → 执行任务 → 再次判断

## ⚡ 只要...就...：重复的秘密

当你需要重复执行某项任务（如打印100次文本）时，while循环是最高效的工具。它的核心就是“只要满足条件，就一直做”。

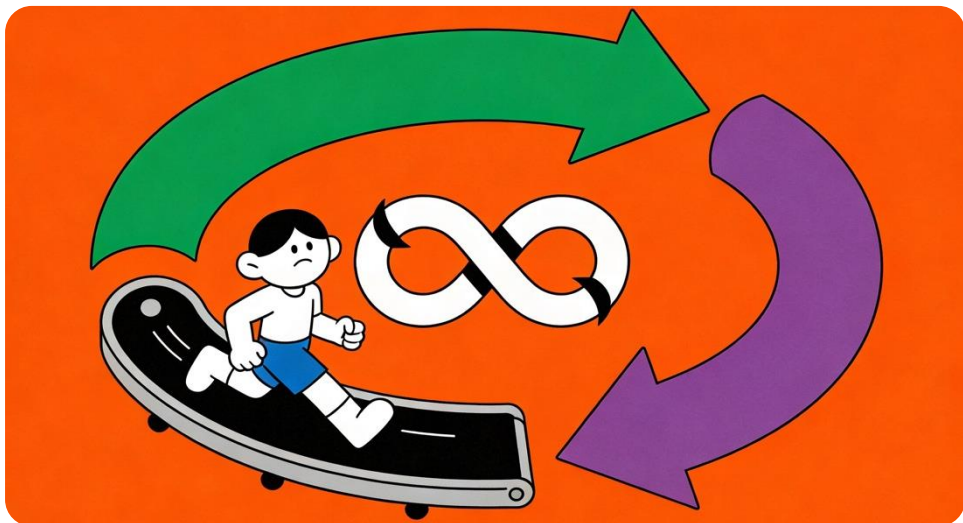
while 条件:

条件满足时，重复执行这里的代码



就像一个不知疲倦的机器人，你给它指令：“只要天没黑，就一直挖地”。它会不停地挖，直到天黑（条件不满足）才停下。

# 死循环：永无止境的噩梦



⚠ 死循环警告：永远跑不到头的跑步机

## ✖ 给机器人一个休息的理由

while循环最大的陷阱是死循环。如果循环条件永远为真，程序将无休止执行，就像让机器人在跑步机上无限奔跑，最终导致系统崩溃。

```
while True: # 无限循环，危险！  
    print("跑...跑...")
```



### 如何避免？

必须在循环内部设置“出口”：通过改变变量值，让条件最终变为 False。

核心法则：写 while 循环时，先想“怎么停下来”，再写循环体！

# 动手实践：发射倒计时！



🚀 准备就绪，开始编码挑战！

## 任务目标：

编写一个倒计时程序，从10倒数到1，最后打印“发射！”  
。重点掌握 While 循环的逻辑。

```
countdown = 10
while countdown > 0:
    print(countdown)
    countdown = countdown - 1
    print("发射！")
```

💡 关键：循环内部必须改变条件变量，否则死循环！

## 05 实战环节



# 打造「数字预言机」游戏：终极挑战

整合变量、逻辑判断与循环知识，亲手实现经典猜数字游戏



### 变量与输入

存储神秘数字与玩家猜测



### 逻辑判断

预言机的核心：大了还是小了？



### 循环机制

让游戏持续直到猜对为止

# 终极挑战：打造你的数字预言机！



## 💡 核心提示

整合循环与条件判断，就是通关关键！

这是一个经典的猜数字游戏。你将把今天学到的变量、循环和条件判断知识整合起来，让电脑成为一位神秘的“预言家”。

## 🎮 游戏核心规则



电脑在 1 到 100 之间随机生成一个秘密数字



玩家输入数字进行猜测，挑战预言家的秘密



电脑根据猜测给出反馈：“猜大了”或“猜小了”



持续猜测直到命中数字，获得胜利，游戏结束

# 第一步：召唤随机数精灵



🌟 编程小魔法师准备就绪

## 🔥 导入随机魔法模块

要让电脑随机想一个数，我们需要借助 Python 的“魔法模块”——**random**。用**import**语句就能把它召唤出来！

```
import random
```

“嘿，Random 精灵，快来帮我生成一个随机数！”

## 第二步：生成秘密数字



### 召唤 Random 精灵

导入random模块后，我们就能借助它的魔法力量，使用randint()函数从指定范围里抽取整数。这就像请来了一位会随机选数的小精灵！

```
secret_number = random.randint(1, 100)
```



数字已存入魔法盒！现在只有电脑知道这个1-100之间的秘密数字是什么。



小提示：randint(a, b) 函数会包括 a 和 b 这两个端点哦！

## 第三步：构建猜数字的循环



💡 核心思路：无限循环 + 条件跳出

因为不确定玩家要猜多少次，所以使用 **while True** 创建无限循环是最佳选择。当玩家猜对时，执行 **break** 语句跳出循环。

```
while True:
    # 1. 获取玩家输入的数字 guess
    # 2. 判断大小并提示；如果猜对了...
    break
```

📢 注意：break 语句必须在 if 判断条件为真时执行

## 第四步：让游戏变得智能 —— 添加判断逻辑



### 1. 获取玩家猜测 (Input)

使用 `input()` 捕获输入，并用 `int()` 转换为整数类型，方便后续比较。



### 2. 逻辑判断 (If/Else)

利用条件分支结构判断猜测结果。猜对则结束游戏，猜错则提示“大了”或“小了”，引导玩家继续。

#### 核心代码演示 (Python)

```
guess=int(input("猜个1-100的数: "))
# --- 判断逻辑 ---
if guess==secret_num:
    print("恭喜，猜对了！")
    break
elif guess>secret_num:
    print("猜大了，再试试！")
```



# 数字预言机：Python 猜数游戏最终版



## 核心逻辑拆解



### 随机数生成

导入 random 模块，生成 1-100 间的秘密数字



### 循环交互机制

While True 无限循环，持续接收玩家猜测输入



### 智能结果判断

If-elif-else 对比大小，给出“大了/小了”提示



### 游戏胜利终止

猜对后执行 break，跳出循环结束游戏



## game\_final.py (Python)

```
# 导入模块与生成秘密数字
import random
secret = random.randint(1, 100)
print("猜个1-100的数字吧！")
# 循环猜数逻辑
while True:
    guess = int(input("你的猜测："))
    if guess == secret:
        print("🎉 恭喜猜对了！")
        break
    elif guess > secret:
        print("猜大了，再小一点")
    else:
        print("猜小了，再大一点")
```

动手运行代码，开启你的数字预言之旅吧！🚀

## 挑战升级：给游戏加上 5 次机会限制！

### 进阶玩法：增加挑战性

游戏太简单了？我们来限制玩家只有 5 次猜测机会。猜错一次机会就减少一次，用完即止！

#### 核心逻辑：

1. 新建变量 `chances = 5` 记录机会。
2. 每次猜错后，`chances = chances - 1`。
3. 循环条件改为：`while chances > 0`。

### Python 代码示例 (pseudocode.py)

```
# 1. 初始化机会变量
chances = 5 # 玩家有5次机会
# 2. 修改循环条件
while chances > 0:
    guess = int(input(f'猜数字({chances}次):'))
    # ... 判断逻辑 ...
    chances -= 1 # 猜错一次机会减一
```

思考：如果再加上函数呢？



# 总结与展示



## 回顾收获

重温今日核心知识点  
夯实编程基础



## 作品展示

欣赏优秀创意成果  
激发创作灵感

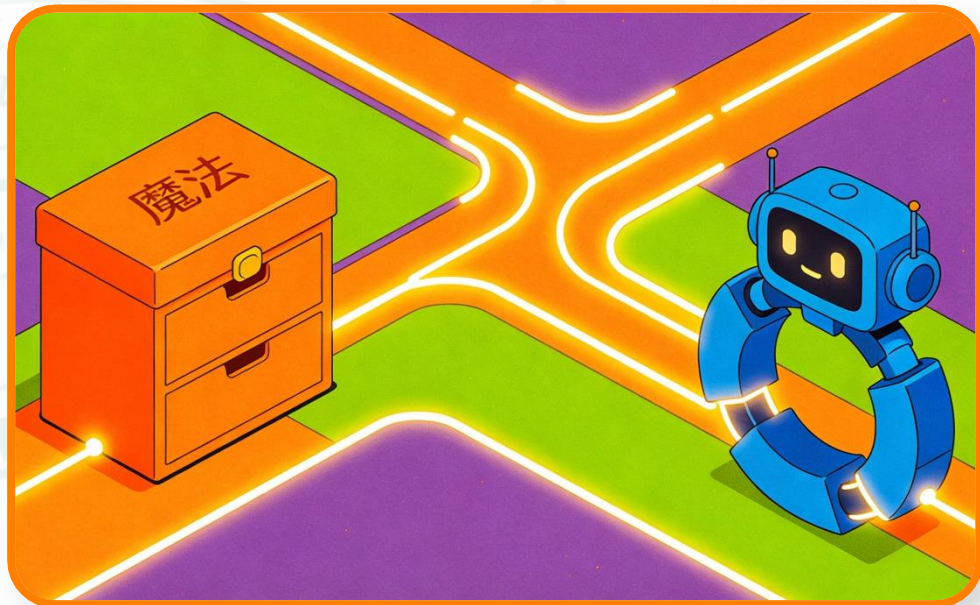


## 总结展望

规划未来学习路径  
开启更多可能

“代码魔法之旅·见证每一个创意时刻”

# 今天我们学到了什么？ | 今日复盘



🌟 代码魔法核心元素图解



## 变量 (Variables)

就像“魔法收纳盒”，专门用来存储数据。



## 函数 (def)

就像“魔法魔法工厂”，给原材料按配方产出。



## 条件判断 (if/else)

程序的“大脑”，让程序学会思考和做选择。



## 循环 (while)

不知疲倦的“机器人”，高效重复执行任务。



我们用这三大基石制作了多个神奇程序，你们每一个人都非常棒！

# 分享与展望：编程之路的起点与延伸



💡 你的代码积木，就是创意的金字塔



## 作品展示时刻

谁愿意展示“数字预言机”？或者分享你对游戏做的有趣修改？



## 课后思考挑战

若想猜1-200或增加提示，应修改代码哪部分？试着挑战更高难度吧！



## 编程初心寄语

伟大的程序员，都是从写出第一行 `print("Hello World")` 开始的。

Keep Coding, Keep Creating —— 保持热爱，奔赴山海

★

# 谢谢观看

请批评指正！

---

期待在未来的编程世界里看到你们更多精彩的作品！

